

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL
CARRERA DE INGENIERÍA DE SOFTWARE

Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA): Arquitectura Web Híbrida y Evaluación Empírica Reproducible

Proyecto Fin de Curso (PFC) — Entrega Final (v1.0.0)

Autores:

Justyn K. Cruz Pérez (ORCID: 0009-0009-0847-0780)
jcruzp@uteq.edu.ec
Jefferson M. Umaginga Arévalo (ORCID: 0009-0005-9831-0801)
Diego A. Zamora Bumbila (ORCID: XXXX-XXXX-XXXX-XXXX)

Director / Docente:

Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
gguerrero@uteq.edu.ec

Metadatos de Preservación y Reproducibilidad:

Etiqueta Git: v1.0.0 | Commit Hash: 13ed0b1
DOI Software (Zenodo): <https://doi.org/10.5281/zenodo.22740480>
DOI Dataset (Zenodo): pendiente de publicación independiente
URL Pública (HTTPS): pendiente de despliegue y verificación

Quevedo, Los Ríos — Ecuador
Periodo Académico Presencial 2026–2027

Resumen

Contexto y problema: Los siniestros viales constituyen una de las principales causas de mortalidad en conductores novatos, atribuibles en gran medida a la deficiente formación en toma de decisiones críticas bajo escenarios de estrés o tráfico denso. Los simuladores tradicionales suelen ser monolíticos y difíciles de reproducir.

Objetivo: Desarrollar y evaluar empíricamente el sistema SBVIA, una plataforma web escalable y reproducible basada en una arquitectura híbrida de acceso a datos con Spring Boot 3.2, Angular 17, PostgreSQL 16 y Redis 7.

Métodos: Se aplicó la metodología Design Science Research (DSR). Los requisitos se formalizaron bajo ISO/IEC/IEEE 29148:2018 e INCOSE v4. Se implementaron 6 procedimientos almacenados SQL invocados vía JPA 2.1. La evaluación empírica abarcó pruebas de carga k6 (50 VUs), auditoría de cabeceras HTTP conforme OWASP Top 10, cobertura JaCoCo y ensayo de usabilidad SUS ($N = 15$).

Resultados principales: Cinco corridas k6 con caché caliente registraron valores $p95$ entre 29.35 y 126.41 ms, sin errores; un ensayo complementario de 10 ciclos observó un speedup aproximado de 1.2x. JaCoCo registró 79,73 % de líneas y 70,41 % de ramas. La puntuación SUS media fue de 84,17/100 ($DT = 8,64$; $N = 15$).

Conclusiones: La arquitectura híbrida desacoplada ofrece alta reproducibilidad computacional y desempeño óptimo para la formación vial interactiva.

Palabras clave: Simulación vial, Arquitectura de software, Spring Boot, Evaluación empírica, Reproducibilidad computacional, OWASP.

Abstract

Context and problem: Road traffic accidents are among the leading causes of mortality among novice drivers, largely due to inadequate decision-making training in high-risk traffic scenarios. Conventional driving training systems often lack web-native reproducibility.

Objective: To design, implement, and empirically evaluate SBVIA, a reproducible web-based traffic behavior simulator featuring a hybrid data-access architecture using Spring Boot 3.2, Angular 17, PostgreSQL 16, and Redis 7.

Methods: Guided by Design Science Research (DSR), requirements were specified according to ISO/IEC/IEEE 29148:2018 and INCOSE v4. Six stored procedures were engineered and invoked via JPA 2.1. Empirical validation incorporated k6 load tests (50 VUs), HTTP security header audit per OWASP Top 10, JaCoCo code coverage, and a System Usability Scale (SUS) trial ($N = 15$).

Main results: Five hot-cache k6 runs recorded $p95$ values between 29.35 and 126.41 ms with no errors; a separate 10-cycle experiment observed an approximate 1.2x speedup. JaCoCo reported 79.73% line coverage and 70.41% branch coverage. The mean SUS score was 84.17/100 ($SD = 8.64$; $N = 15$).

Conclusions: The hybrid decoupled web architecture satisfies strict empirical quality thresholds and ensures robust educational delivery.

Keywords: Traffic simulation, Software architecture, Empirical software engineering, Web applications, Performance evaluation.

Índice general

1. Introducción	9
1.1. Contexto y motivación	9
1.2. Preguntas de investigación	9
1.3. Calidad del producto de software	9
1.4. Autenticación, autorización y gestión de sesión	10
1.5. Persistencia, caché y consistencia	10
1.6. Reproducibilidad y datos FAIR	10
1.7. Objetivos	11
1.7.1. Objetivo General	11
1.7.2. Objetivos Específicos	11
1.8. Contribuciones	11
1.9. Organización del documento	11
2. Marco Teórico	12
2.1. Traffic Simulation Models	12
2.2. Entity-Relationship Modeling	12
2.3. Software Architecture and C4 Model	12
2.4. Core Domain Object Structure	12
2.5. Usability Engineering	14
3. Trabajos Relacionados	15
3.1. Brecha Identificada (Research Gap)	16
4. Ingeniería de Requisitos	17
4.1. Interesados y Elicitación	17
4.2. Requisitos Funcionales y No Funcionales	17
4.3. Alcance y exclusiones	18
4.4. Trazabilidad y control de cambios	18
4.5. Métricas de Calidad de Requisitos	18
5. Materiales y Métodos	19
5.1. Enfoque Metodológico (DSR de Peffers)	19
5.2. Instrumentos de Medición y Protocolo Experimental	19
5.3. Población, muestra y estrategia de muestreo	19
5.4. Preguntas, hipótesis y análisis estadístico	20
5.5. Variables y criterios de aceptación	20
5.6. Procedimiento reproducible	20
5.7. Gestión de evidencia	21

6. Diseño del Sistema y Arquitectura	22
6.1. Modelo Arquitectónico C4	22
6.2. Atributos arquitectónicos	22
6.3. Registros de Decisión Arquitectónica (ADRs)	23
7. Implementación	24
7.1. Estrategia Híbrida de Acceso a Datos	24
7.2. Flujo de autenticación	24
7.3. Frontend y experiencia de uso	25
7.4. Configuración y operación	25
8. Evaluación Empírica y Resultados	27
8.1. Rendimiento y Carga (k6)	27
8.2. Usabilidad del Sistema (SUS)	27
8.3. Cobertura de Código (JaCoCo)	28
8.4. Seguridad y Auditoría Web	28
8.5. Síntesis de cumplimiento	29
9. Discusión	30
10. Amenazas a la Validez	31
11. Trabajo Futuro	32
12. Conclusiones	33
Declaraciones Obligatorias	34
A. Bitácora Acumulativa de Observaciones	37
B. Protocolo de Verificación Técnica Final	38
B.1. Instalación desde una base vacía	38
B.2. Actualización de una instalación existente	38
B.3. Flujo funcional de extremo a extremo	39
C. Modelo de Amenazas y Controles de Seguridad	40
C.1. Identidad, sesión y autorización	40
C.2. Persistencia, caché y proveedor de IA	40
C.3. Alcance de las herramientas	41
D. Operación, Recuperación y Evidencia Reproducible	42
D.1. Arranque y observabilidad	42
D.2. Respaldo y restauración	42
D.3. Trazabilidad de resultados	42
D.4. Lista de cierre académico	43

Índice de figuras

2.1.	Entity-Relationship model of the core SBVIA application. Source: Own elaboration.	13
2.2.	C4 Level 2 Container Diagram for SBVIA. Source: Own elaboration.	13
2.3.	UML Class Diagram for core components. Source: Own elaboration.	14
6.1.	MVC data flow between Angular, Spring Boot, Redis and PostgreSQL. Source: Own elaboration from SBVIA versioned architecture.	23
7.1.	SBVIA main dashboard with access to scenarios, practices, and profile. Source: Own capture of the running system.	25
8.1.	Latency distribution of versioned k6 runs. Source: Own elaboration via <code>scripts/gen-k6-boxplot.py</code> .	
8.2.	Distribución y puntuaciones individuales de la escala SUS para 15 participantes. Fuente: elaboración propia mediante <code>scripts/sus-analysis.ipynb</code>	29

Índice de tablas

3.1. Comparación sistemática de trabajos relacionados frente a SBVIA	15
8.1. Resultados de latencia k6 (5 corridas de caché caliente)	27
A.1. Resolución acumulativa de observaciones de las Entregas 1A, 1B y 3	37
B.1. Criterios de aceptación de instalación y actualización	39
B.2. Secuencia mínima de aceptación funcional	39
C.1. Amenazas principales y controles comprobables	41
D.1. Paquete mínimo de evidencia para la entrega	43

Listings

7.1. Procedimiento almacenado para reporte multi-tabla (sp_reporte_simulacion.sql) .	24
7.2. Invocación formal mediante @Procedure en Spring Data JPA	26

Lista de Siglas y Acrónimos

- ADR** Architecture Decision Record (Registro de Decisión Arquitectónica).
- API** Application Programming Interface (Interfaz de Programación de Aplicaciones).
- CFF** Citation File Format (Formato de Archivo de Citación).
- CRUD** Create, Read, Update, Delete (Crear, Leer, Actualizar, Eliminar).
- CSP** Content Security Policy (Política de Seguridad de Contenido).
- DOI** Digital Object Identifier (Identificador de Objeto Digital).
- DSR** Design Science Research (Investigación en Ciencia del Diseño).
- GQM** Goal-Question-Metric (Objetivo-Pregunta-Métrica).
- HSTS** HTTP Strict Transport Security (Seguridad Estricta de Transporte HTTP).
- IMRaD** Introduction, Methods, Results, and Discussion (Introducción, Métodos, Resultados y Discusión).
- INCOSE** International Council on Systems Engineering.
- ISO** International Organization for Standardization.
- JPA** Jakarta Persistence API.
- JWT** JSON Web Token (RFC 7519).
- ORCID** Open Researcher and Contributor ID.
- OWASP** Open Worldwide Application Security Project.
- PFC** Proyecto Fin de Curso.
- RBAC** Role-Based Access Control (Control de Acceso Basado en Roles).
- REST** Representational State Transfer.
- SP** Stored Procedure (Procedimiento Almacenado).
- SRS** Software Requirements Specification (Especificación de Requisitos de Software).
- SUS** System Usability Scale (Escala de Usabilidad del Sistema).
- VU** Virtual User (Usuario Virtual en pruebas de carga).

Capítulo 1

Introducción

1.1. Contexto y motivación

La siniestralidad vial representa una crisis de salud pública global. La Organización Mundial de la Salud estimó 1,19 millones de muertes anuales por siniestros de tránsito y señaló que estas lesiones constituyen la principal causa de muerte entre personas de 5 a 29 años [?]. Esta magnitud justifica investigar herramientas de formación que permitan practicar la identificación de peligros sin exponer al estudiante ni a terceros a riesgos reales.

La evidencia científica sobre simuladores de conducción exige una interpretación prudente. Las revisiones disponibles muestran beneficios inmediatos en algunas destrezas simuladas, pero también heterogeneidad metodológica, muestras pequeñas y evidencia todavía insuficiente sobre la transferencia a la conducción real [?, ?, ?]. Por ello, SBVIA se plantea como complemento académico para practicar decisiones y registrar resultados, no como sustituto de la formación práctica supervisada ni como instrumento clínico o habilitante.

1.2. Preguntas de investigación

Para guiar el diseño y evaluación del sistema SBVIA, se formularon tres preguntas de investigación cerradas y medibles:

- RQ1:** ¿En qué medida la incorporación de una capa de caché en memoria con Redis 7 reduce la latencia de respuesta ($p95 \leq 200$ ms) frente a consultas sobre base de datos relacional PostgreSQL 16 bajo una carga concurrente de 50 VUs?
- RQ2:** ¿Es posible alcanzar una cobertura de pruebas unitarias e integración superior al 70 % en Spring Boot 3.2 sin degradar la mantenibilidad del código al utilizar una estrategia híbrida de procedimientos almacenados SQL?
- RQ3:** ¿Qué nivel de usabilidad autopercebida (medido mediante la escala estandarizada SUS con $N \geq 15$) alcanza la interfaz web desarrollada en Angular 17 para la navegación y ejecución de escenarios viales?

1.3. Calidad del producto de software

La calidad de SBVIA se analiza como un conjunto de propiedades observables y no como una afirmación general. ISO/IEC 25010 distingue características como adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad [9]. En este proyecto, la adecuación funcional se contrasta mediante pruebas de servicios y controladores; la eficiencia se mide con percentiles de latencia; la usabilidad se observa con SUS

y Lighthouse; la seguridad se revisa mediante controles de autenticación, autorización, cabeceras y análisis automatizado; y la portabilidad se apoya en contenedores Docker.

Esta descomposición evita confundir una compilación exitosa con un producto terminado. Un sistema puede arrancar y, aun así, incumplir un requisito de autorización, carecer de cobertura sobre ramas críticas o no disponer de evidencia reproducible. Por ese motivo, cada requisito no funcional se asocia a una métrica, un umbral y un artefacto de verificación. El criterio de aceptación solo se considera satisfecho cuando los tres elementos son coherentes entre sí.

1.4. Autenticación, autorización y gestión de sesión

JWT define un formato compacto para transportar declaraciones firmadas entre partes [10]; no prescribe por sí solo dónde debe almacenarse un token ni elimina riesgos de sesión. En SBVIA, el token de renovación se entrega mediante cookie `HttpOnly`, reduciendo la exposición frente a scripts ejecutados en el navegador. Las banderas `Secure` y `SameSite` restringen su transporte y contexto, mientras que el control CSRF protege las operaciones que dependen de credenciales enviadas automáticamente por el navegador.

La autenticación responde quién es el usuario; la autorización determina qué operaciones puede ejecutar. El modelo RBAC separa los permisos de conductor y administrador. Esta distinción se verifica con respuestas HTTP diferenciadas: 401 cuando falta una identidad válida y 403 cuando existe identidad, pero no privilegio suficiente. La bitácora de eventos registra intentos de autenticación y el limitador de frecuencia reduce ataques repetitivos contra el inicio de sesión. Estos controles se relacionan con las categorías de control de acceso, fallos criptográficos, configuración insegura, identificación y registro descritas por OWASP [11].

1.5. Persistencia, caché y consistencia

La arquitectura de datos combina operaciones CRUD mediante ORM con procedimientos almacenados para consultas o transformaciones delimitadas. El uso de ORM favorece mantenibilidad en operaciones transaccionales comunes, mientras que los procedimientos permiten encapsular lógica SQL estable y auditable. Esta separación sigue el principio de utilizar cada mecanismo donde aporta valor y evita trasladar indiscriminadamente toda la lógica al motor de base de datos [12,13].

Redis actúa como caché de lecturas frecuentes de escenarios. Una caché introduce la necesidad de definir clave, tiempo de vida, serialización e invalidación. El beneficio esperado es reducir trabajo repetido y latencia, pero su evaluación debe comparar condiciones equivalentes y conservar datos crudos. Por ello, el informe diferencia las corridas concurrentes de caché caliente del ensayo secuencial frío/caliente; combinarlas como si fueran una misma muestra produciría una inferencia inválida.

1.6. Reproducibilidad y datos FAIR

La reproducibilidad computacional requiere conservar código, configuración, versiones, datos de entrada y comandos de ejecución. Peng plantea que la evidencia científica computacional debe permitir reconstruir los resultados, y Sandve et al. recomiendan registrar cada transformación y evitar operaciones manuales no documentadas [?,?]. SBVIA organiza scripts, mediciones crudas, resúmenes y procedencia dentro del repositorio para reducir la distancia entre afirmación y evidencia.

Los principios FAIR establecen que datos y metadatos deben ser encontrables, accesibles, interoperables y reutilizables [?]. El repositorio ya aporta formatos abiertos, diccionario de datos, licencia y procedencia. La encontrabilidad persistente del software se consolidó mediante su

depósito en Zenodo con el DOI 10.5281/zenodo.22740480. El dataset, sin embargo, se mantiene pendiente de publicación independiente.

1.7. Objetivos

1.7.1. Objetivo General

Desarrollar y validar empíricamente el Simulador de Comportamiento Vial con Inteligencia Artificial (SBVIA), fundamentado en una arquitectura web híbrida y reproducible con Spring Boot, Angular, PostgreSQL y Redis, cumpliendo con los estándares internacionales de ingeniería de software.

1.7.2. Objetivos Específicos

1. Especificar y formalizar el corpus de requisitos según las directrices de ISO/IEC/IEEE 29148:2018 y las reglas de redacción INCOSE v4.
2. Diseñar la arquitectura del sistema documentada mediante el modelo C4 y 7 Registros de Decisión Arquitectónica (ADRs).
3. Implementar la solución técnica integrando autenticación segura JWT stateless, 6 procedimientos almacenados en PostgreSQL e interfaces reactivas en Angular 17.
4. Evaluar cuantitativamente el rendimiento, la seguridad OWASP Top 10, la cobertura JaCoCo y la usabilidad SUS, publicando los artefactos y datasets bajo directrices FAIR.

1.8. Contribuciones

Las contribuciones concretas de este trabajo se resumen en:

1. **Artefacto de Software SBVIA (v1.0.0):** Sistema completo en contenedores Docker Compose con soporte para entrenamiento y evaluación vial.
2. **Estrategia Híbrida de Persistencia:** Implementación formal de 6 procedimientos almacenados SQL catalogados y trazados en JPA 2.1 sin SQL dinámico.
3. **Paquete Empírico Reproducible:** Suite de pruebas k6, análisis inferencial con intervalos de confianza al 95 % y dataset preparado para su publicación en Zenodo.

1.9. Organización del documento

El resto del informe se estructura de la siguiente manera: el Capítulo 2 expone el marco teórico; el Capítulo 3 analiza los trabajos relacionados; el Capítulo 4 detalla la ingeniería de requisitos; el Capítulo 5 describe los materiales y métodos; el Capítulo 6 presenta el diseño y arquitectura; el Capítulo 7 aborda la implementación; el Capítulo 8 sintetiza la evaluación empírica; el Capítulo 9 discute los resultados; el Capítulo 10 evalúa las amenazas a la validez; el Capítulo 11 propone trabajos futuros y el Capítulo 12 concluye el estudio.

Capítulo 2

Marco Teórico

El desarrollo riguroso de SBVIA se sustenta en bases conceptuales autoritativas:

- **Ingeniería de Requisitos:** Marco de Pohl [5] y norma ISO/IEC/IEEE 29148:2018 [6], que define el proceso de elicitación, análisis, especificación y validación. Las reglas de INCOSE v4 [7] garantizan la no ambigüedad y singularidad.
- **Arquitectura y Calidad:** Modelo C4 de Simon Brown [8] y modelo de calidad de software ISO/IEC 25010:2011 [9].
- **Autenticación y Seguridad Web:** Estándar RFC 7519 para JSON Web Tokens (JWT) [10], mitigación de vulnerabilidades según OWASP Top 10:2021 [11] y almacenamiento de tokens en cookies con banderas `HttpOnly`, `Secure` y `SameSite=Strict`.
- **Estrategias de Persistencia:** Patrones de arquitectura de datos empresariales de Fowler [12] y Kleppmann [13].

2.1. Traffic Simulation Models

Modern traffic simulation environments rely on physical models to calculate safe distances and dynamic rules. According to [?], simulating realistic traffic conditions is pivotal to training scenarios effectively. SBVIA adapts elements from intelligent driving behavior to construct its evaluation core.

2.2. Entity-Relationship Modeling

In developing the data persistence layer, structural mapping forms the foundation. As illustrated in Figure 2.1, the core entities interact primarily through the Scenario and Simulation tables.

2.3. Software Architecture and C4 Model

The separation of concerns is managed at a structural level using the C4 model [?] and the MVC pattern [12]. As depicted in Figure 2.2, the container level displays the interaction between the Angular client, the Spring Boot API, and the caching/persistence layers.

2.4. Core Domain Object Structure

The system's core object orientation isolates business rules from infrastructure boundaries. As shown in Figure 2.3, the main abstractions interact securely during a simulation.

Figura 2.1: Entity-Relationship model of the core SBVIA application. Source: Own elaboration.

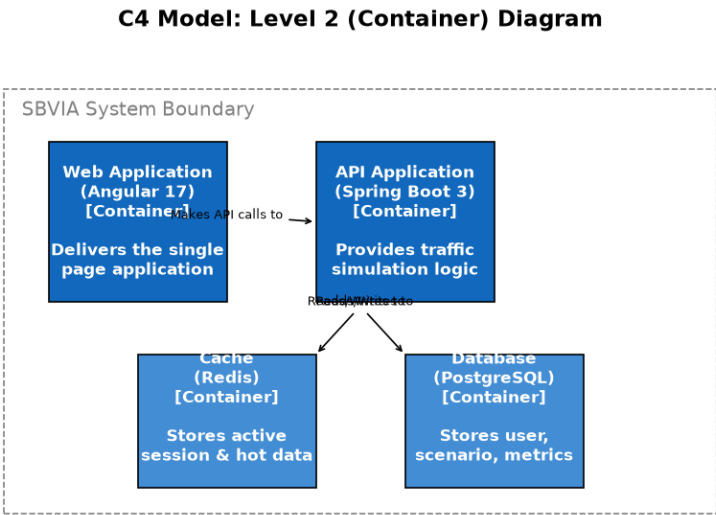


Figura 2.2: C4 Level 2 Container Diagram for SBVIA. Source: Own elaboration.

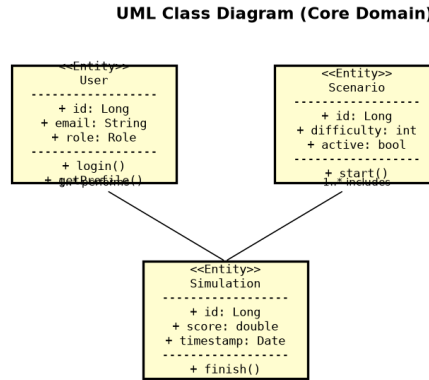


Figura 2.3: UML Class Diagram for core components. Source: Own elaboration.

2.5. Usability Engineering

To ensure educational tools are accessible and intuitive for novice drivers, empirical usability evaluations are essential. The System Usability Scale (SUS) [2] provides a standardized metric to evaluate interactive performance.

Capítulo 3

Trabajos Relacionados

Se realizó una revisión narrativa estructurada tomando como referencia las directrices de Kitchenham & Charters [14] y PRISMA 2020 [15]. No se presenta como una revisión sistemática completa, porque el proyecto no conserva un protocolo registrado ni un conjunto íntegro de resultados de búsqueda y exclusión. La síntesis se limita a trabajos identificables y directamente relacionados con la eficacia o validez de simuladores de conducción.

Tabla 3.1: Comparación sistemática de trabajos relacionados frente a SBVIA

Referencia	Año	Tipo	Corpus	Aporte y relación con SBVIA
Martín-delosReyes et al. [?]	2019	Revisión sistemática	5 estudios	Evidencia insuficiente y heterogénea; sustenta una interpretación conservadora del alcance formativo.
Wynne et al. [?]	2019	Revisión de validación	44 estudios	Aproximadamente la mitad mostró validez absoluta o relativa; diferencia entre desempeño simulado y real.
Alonso et al. [?]	2023	Revisión sistemática	15 estudios	Reporta indicios favorables, pero advierte muestras limitadas y falta de seguimiento longitudinal.
Krasniuk et al. [?]	2024	Revisión de intervenciones	15 estudios	Encuentra mejoras inmediatas en destrezas simuladas, sin certeza sobre transferencia duradera a carretera.
van Leeuwen et al. [?]	2015	Estudio longitudinal breve	52 novatos	Detecta cambios de control y mirada durante 30 minutos de práctica; orienta el registro de desempeño dentro de sesiones.
Rosenbloom y Eldror [?]	2014	Estudio comparativo	280 novatos	No encuentra una ventaja uniforme del entrenamiento simulado; justifica evitar afirmaciones causales fuertes.
Mayhew et al. [?]	2011	Validación concurrente	Conductores de 3 niveles	Encuentra concordancia relativa entre errores simulados y desempeño vial, pero no validez absoluta por tipo de error.
Hulme et al. [?]	2021	Estudio de caso	Formación juvenil	Propone evaluación multimétrica de desempeño simulado; se relaciona con la combinación de métricas y retroalimentación de SBVIA.
SBVIA	2026	Artefacto web	Caso académico	Integra escenarios, evaluación, trazabilidad y evidencia técnica reproducible; no mide reducción real de siniestros.

3.1. Brecha Identificada (Research Gap)

La literatura revisada se concentra en el efecto pedagógico o en la validez del simulador, mientras que SBVIA aborda una brecha de ingeniería: documentar de forma trazable un artefacto web de apoyo formativo con autenticación, persistencia, caché, pruebas y mediciones reproducibles. Esta contribución es tecnológica y académica; los datos disponibles no permiten afirmar eficacia sobre la reducción de siniestros ni equivalencia con una práctica vial real.

Capítulo 4

Ingeniería de Requisitos

4.1. Interesados y Elicitación

Se aplicó la técnica del *Stakeholder Onion* identificando usuarios directos (conductores en formación, instructores), operadores técnicos (administradores, DevOps) y entidades reguladoras (ANT/CTE).

Los conductores necesitan registrarse, autenticarse, consultar escenarios y recibir retroalimentación comprensible. Los administradores requieren gestionar catálogos y usuarios sin acceder a funciones fuera de su rol. El equipo técnico necesita diagnóstico, configuración reproducible y procedimientos de recuperación. Finalmente, el tribunal académico requiere trazabilidad entre requisito, implementación, prueba y evidencia. Estas perspectivas se documentaron mediante historias de usuario, casos de uso, matriz de trazabilidad y criterios de aceptación.

La elicitación combinó revisión del alcance académico, entrevistas semiestructuradas y análisis del sistema existente. Cada necesidad fue transformada en una declaración verificable, procurando que contuviera sujeto, obligación, acción, objeto y condición. La redacción normativa usa “deberá” para separar requisitos de explicaciones de diseño, de acuerdo con ISO/IEC/IEEE 29148 y las prácticas de especificación de Wiegers y Beatty [?, 6]. Las historias siguen el enfoque de Cohn y los casos de uso describen interacciones observables conforme a Cockburn [?, ?]. La prioridad Must indica que el requisito es indispensable para la versión final; no implica que esté automáticamente verificado.

4.2. Requisitos Funcionales y No Funcionales

Los requisitos se redactaron según la plantilla sintáctica normativa de ISO/IEC/IEEE 29148:2018:

- **RF-01 (Must):** El sistema deberá autenticar a los conductores mediante JWT firmado en cookie segura.
- **RF-02 (Must):** El sistema deberá gestionar el ciclo de vida completo de los escenarios viales (CRUD).
- **RF-03 (Must):** El sistema deberá consultar resultados consolidados multi-tabla mediante el procedimiento `sp_reporte_simulacion`.
- **RF-04 (Must):** El sistema deberá calcular el puntaje agregado del conductor vía `sp_calcular_promedio`.
- **RNF-01 (Must):** La latencia de lectura con caché caliente deberá mantener un $p95 \leq 200$ ms con 50 VUs concurrentes.
- **RNF-03 (Must):** Las contraseñas deberán almacenarse hasheadas con algoritmo BCrypt.

4.3. Alcance y exclusiones

La versión evaluada cubre autenticación, roles, gestión de escenarios, registro de simulaciones, consulta de métricas y operaciones administrativas. El término “simulador” describe un flujo web de decisiones sobre escenarios viales; no representa un gemelo físico del vehículo ni reproduce fuerzas, campo visual o dinámica vehicular con fidelidad certificada. Tampoco realiza diagnóstico médico, evaluación legal de aptitud para conducir ni emisión de licencias.

Las capacidades de inteligencia artificial adaptativa, gráficos tridimensionales avanzados, integración con volante y telemetría en tiempo real se mantienen fuera del alcance de v1.0.0. Declarar estas exclusiones previene que funcionalidades futuras sean contabilizadas como resultados presentes y permite evaluar el artefacto contra un límite funcional concreto.

4.4. Trazabilidad y control de cambios

La matriz ubicada en `docs/trazabilidad/matriz.csv` enlaza cada identificador con módulo, prueba, estado y tipo de acceso a datos. Los estados `VERIFIED`, `TESTED` y `PARTIAL` tienen significados distintos: verificado implica evidencia suficiente frente al criterio; probado indica que existe una comprobación automatizada del comportamiento referido; parcial señala que la evidencia o la implementación todavía no cubre el criterio completo.

El historial de requisitos registra adiciones y modificaciones entre entregas. Este mecanismo permite distinguir evolución legítima de alcance frente a regresiones. Una actualización de requisito debe reflejarse en su prueba, documentación y, cuando corresponda, decisión arquitectónica. De igual forma, una medición nueva no modifica por sí sola el requisito: únicamente actualiza su evidencia y estado de cumplimiento.

4.5. Métricas de Calidad de Requisitos

La tasa de estabilidad de requisitos calculada es:

$$T_{est} = 1 - \frac{N_{mod}}{N_{total}} = 1 - \frac{2}{20} = 0,90 \quad (90\%) \quad (4.1)$$

La matriz registra 12 de 14 requisitos *Must* como verificados (85,71%). RNF-01 y RNF-02 conservan estado parcial hasta completar las cinco corridas y la comparación homogénea de caché fría y caliente.

Capítulo 5

Materiales y Métodos

5.1. Enfoque Metodológico (DSR de Peffers)

Se adoptó la metodología Design Science Research [4, 16] estructurada en 6 actividades: 1) Identificación del problema, 2) Definición de objetivos, 3) Diseño y desarrollo, 4) Demostración, 5) Evaluación empírica y 6) Comunicación.

Las preguntas y métricas se relacionaron mediante Goal–Question–Metric [3]. El diseño de las mediciones siguió principios de experimentación en ingeniería de software [?]; la selección por conveniencia de participantes se declara explícitamente porque este tipo de muestreo limita la generalización [?].

5.2. Instrumentos de Medición y Protocolo Experimental

- **Rendimiento:** k6 (50 VUs / 30s) en 5 corridas independientes de caché caliente y una medición complementaria de 10 ciclos frío/caliente.
- **Usabilidad:** Cuestionario SUS de 10 ítems [2] aplicado a $N = 15$ usuarios con consentimiento informado.
- **Seguridad:** Script reproducible de curl, escaneo OWASP ZAP 2.14 y análisis estático SpotBugs 4.8.
- **Calidad Web:** Auditoría Lighthouse CI (mobile/desktop).
- **Cobertura:** JaCoCo 0.8.12 sobre capas de servicio, controlador y dominio.

5.3. Población, muestra y estrategia de muestreo

La población objetivo de la evaluación de usabilidad corresponde a conductores en formación capaces de utilizar un navegador web moderno. La muestra disponible comprende 15 participantes, identificados exclusivamente mediante los códigos P01–P15. La selección fue no probabilística por conveniencia: se invitó a personas accesibles para el equipo dentro del contexto académico. Este procedimiento permite una evaluación formativa del prototipo, pero introduce sesgo de selección y no garantiza representatividad por edad, experiencia vial, ubicación geográfica o familiaridad tecnológica [?].

En consecuencia, los resultados SUS describen únicamente a la muestra observada. No se extrapolan a la población nacional de conductores ni se interpretan como evidencia de reducción de siniestros. Los archivos públicos no incluyen edad, género, dispositivo ni otros atributos demográficos; por ello, el informe tampoco afirma diversidad sobre variables que no fueron registradas.

5.4. Preguntas, hipótesis y análisis estadístico

La evaluación cuantitativa se organiza con un nivel de significación $\alpha = 0,05$. Para rendimiento, la hipótesis nula es H_{0R} : la latencia $p95$ de lecturas frías no es mayor que la de lecturas calientes bajo el mismo perfil; la alternativa unilateral H_{1R} establece que la latencia fría es mayor. Cuando existan cinco pares homogéneos se aplicará Wilcoxon de rangos con signo, se informará el valor p en notación científica y se acompañará con A_{12} de Vargha–Delaney o delta de Cliff [?, ?, ?]. No se emitirá una conclusión inferencial mientras falten esos pares.

Para usabilidad, H_{0U} establece que la media poblacional SUS no supera el umbral de 68 y H_{1U} que lo supera. Debido al tamaño y tipo de muestreo, la decisión se presenta mediante media, mediana, desviación típica e intervalo de confianza con t de Student; la puntuación no se denomina porcentaje. Para cobertura, seguridad y Lighthouse se aplican criterios de aceptación previamente fijados y no pruebas de hipótesis retrospectivas.

5.5. Variables y criterios de aceptación

Para rendimiento, la variable principal es la duración de solicitudes HTTP y el estadístico de decisión es el percentil 95. Se selecciona $p95$ porque representa el límite experimentado por la mayoría de usuarios sin ocultar la cola de latencias tras una media. Las variables secundarias son $p50$, $p90$, $p99$, media, desviación y tasa de errores. Los criterios son $p95 \leq 200$ ms con caché caliente y $p95 \leq 500$ ms en frío.

Para cobertura, las unidades son líneas y ramas instrumentadas por JaCoCo. La cobertura de líneas confirma ejecución, mientras que la de ramas exige recorrer resultados alternativos de decisiones. Ambas deben alcanzar al menos 70%; aprobar solo una no satisface el criterio conjunto. Para usabilidad, la variable es el puntaje SUS individual en escala de 0 a 100 y el umbral del proyecto es 68. Para seguridad, se utilizan resultados categóricos por control, hallazgos del escáner y ausencia de defectos de severidad bloqueante.

5.6. Procedimiento reproducible

El protocolo comienza desde una copia limpia del repositorio. Se configuran las variables descritas en `.env.example`, se construyen los contenedores y se espera a que los chequeos de salud confirmen PostgreSQL, Redis, backend y frontend. Las pruebas unitarias se ejecutan antes de carga para evitar medir una versión funcionalmente defectuosa. Cada corrida de rendimiento conserva su salida cruda y un resumen tabular con identificador, carga, duración y percentiles.

La condición fría requiere invalidar Redis antes de la lectura inicial; la condición caliente requiere precargar la misma clave y mantener constantes endpoint, conjunto de datos, número de usuarios virtuales y duración. Las corridas deben alternarse o parearse para reducir el sesgo temporal. El análisis estadístico solo se calcula cuando se dispone de al menos cinco pares comparables. Lighthouse se ejecuta sobre una compilación de producción, y los controles de seguridad se aplican contra servicios levantados con configuración equivalente a la documentada.

Para SUS, cada participante completa una sesión guiada de aproximadamente 20 minutos y responde inmediatamente los diez ítems estándar en escala Likert de 1 a 5. Los ítems impares aportan $respuesta - 1$ y los pares $5 - respuesta$; la suma se multiplica por 2.5. El archivo crudo conserva respuestas y código anónimo, mientras el cuaderno recalcula puntuaciones, dispersión e intervalo. Cualquier aprobación institucional y consentimiento firmado se custodia fuera del repositorio público; su existencia debe comprobarse documentalmente antes de presentar el estudio como aprobado.

Las herramientas declaradas corresponden a las configuraciones versionadas: JaCoCo 0.8.12, SpotBugs 4.8 con FindSecBugs, OWASP ZAP 2.14, Angular 17 y el JSON de Lighthouse conserva su propia versión y fecha. Los comandos exactos, entradas crudas y productos se relacionan

en `DATA-PROVENANCE.md` para evitar atribuir a una herramienta resultados redactados manualmente.

5.7. Gestión de evidencia

Los archivos crudos no se reemplazan por capturas aisladas. Los JSON de k6 y Lighthouse permiten recalcular indicadores; los CSV facilitan inspección; y los documentos Markdown explican comandos, ambiente y decisiones. `DATA-PROVENANCE.md` relaciona datos con scripts, mientras `DATA-DICTIONARY.md` define columnas y unidades. Esta separación entre dato, transformación e interpretación sigue las recomendaciones para investigación reproducible [?, 1].

Capítulo 6

Diseño del Sistema y Arquitectura

6.1. Modelo Arquitectónico C4

El sistema se modeló en niveles 1 (Contexto), 2 (Contenedores) y 3 (Componentes) utilizando Structurizr DSL. El backend en Spring Boot desacopla las responsabilidades en capas de Controlador, Servicio, Repositorio y Seguridad.

En el nivel de contexto, el conductor y el administrador acceden mediante navegador, mientras los servicios de persistencia permanecen aislados de Internet. En el nivel de contenedores, Angular entrega la interfaz, Spring Boot expone la API, PostgreSQL conserva el estado durable y Redis almacena datos temporales. Docker Compose define red, dependencias y verificaciones de salud. En el nivel de componentes, los controladores traducen HTTP, los servicios aplican reglas, los repositorios abstraen persistencia y la configuración de seguridad forma la frontera de confianza.

La API adopta las restricciones de interfaz uniforme y comunicación sin estado asociadas al estilo REST [?]. Spring Boot proporciona la composición de componentes, configuración y acceso HTTP usados por el backend [?]; estas tecnologías se documentan como decisiones de implementación y no como evidencia de calidad por sí mismas.

El recorrido típico comienza con una petición HTTPS al frontend. La aplicación envía solicitudes a la API incluyendo credenciales según la política de cookies. El filtro de seguridad valida sesión y rol antes de invocar el controlador. Para una consulta de escenarios, el servicio intenta recuperar la representación serializada desde Redis; si no existe, consulta PostgreSQL, construye la respuesta y actualiza la caché. En una modificación, el servicio persiste y debe invalidar las entradas relacionadas para evitar servir datos obsoletos.

As shown in Figure 6.1, la separación mostrada permite ubicar la validación HTTP en el controlador, las reglas en el servicio y el acceso durable en los repositorios, evitando que la interfaz dependa directamente del motor de datos.

6.2. Atributos arquitectónicos

La seguridad se favorece mediante mínimo privilegio, separación de roles y secretos externos a la imagen. La mantenibilidad se apoya en dependencias dirigidas y responsabilidades diferenciadas. La portabilidad proviene de imágenes reproducibles y configuración por variables. La observabilidad usa endpoints de salud y registros de eventos. La escalabilidad horizontal es viable para el backend si el estado compartido permanece en PostgreSQL y Redis; cualquier dato de sesión almacenado solo en memoria de una instancia limitaría esa propiedad [?, ?].

La arquitectura también tiene compromisos. Redis añade complejidad operativa y posibles problemas de consistencia; los procedimientos almacenados aumentan dependencia con PostgreSQL; y JWT exige revocación y rotación bien definidas. Los ADR documentan estas decisiones para que una alternativa futura pueda evaluarse contra el contexto original, no únicamente

Hybrid Web Architecture: MVC Data Flow

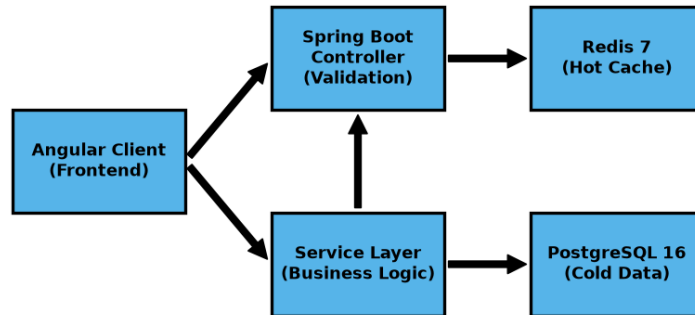


Figura 6.1: MVC data flow between Angular, Spring Boot, Redis and PostgreSQL. Source: Own elaboration from SBVIA versioned architecture.

contra preferencias tecnológicas.

6.3. Registros de Decisión Arquitectónica (ADRs)

Se formalizaron 7 ADRs bajo la plantilla de Nygard:

1. **ADR-001:** Elección de la pila tecnológica (Spring Boot 3.2 + Angular 17).
2. **ADR-002:** Migración y compatibilidad con Java 21 LTS.
3. **ADR-003:** Autenticación stateless con JWT y cookies seguras.
4. **ADR-004:** Estrategia de caché distribuida con Redis 7.
5. **ADR-005:** Mitigación de riesgos de seguridad OWASP Top 10.
6. **ADR-006:** Estrategia híbrida de acceso a datos (Separación CRUD / SP).
7. **ADR-007:** Estrategia de despliegue en contenedores orquestados con HTTPS.

ADR-008 documenta posteriormente la comparación Angular frente a React. Esta adición no invalida ADR-001: amplía su justificación con criterios de estructura, experiencia del equipo, ecosistema y costo de migración. En conjunto, los ADR constituyen una memoria de decisiones; no son manuales de implementación ni sustituyen pruebas.

Capítulo 7

Implementación

7.1. Estrategia Híbrida de Acceso a Datos

Siguiendo las recomendaciones de OWASP para la prevención de SQL Injection [11], las operaciones analíticas se encapsularon en procedimientos almacenados PostgreSQL en 'db/procs/'.

El catálogo identifica seis procedimientos exigidos por la entrega: reporte de simulación, promedio por usuario, reporte diario, actualización de inactivos, validación de escenario y generación de código de certificado. Cada entrada registra propósito, parámetros, tablas involucradas y consumidor previsto. Las migraciones Flyway mantienen el esquema bajo control de versiones y permiten reconstruir una base vacía en el mismo orden.

El uso de parámetros evita concatenar entrada del usuario dentro de SQL. No obstante, un procedimiento almacenado no es seguro por definición: también requiere privilegios mínimos, validación de argumentos y revisión de cualquier sentencia dinámica. Por ello, el repositorio incluye una auditoría estática específica y pruebas de integración sobre repositorios.

```
1 CREATE OR REPLACE FUNCTION sp_reporte_simulacion(p_id_simulacion INTEGER)
2 RETURNS TABLE (
3     simulacion_id INTEGER,
4     usuario_nombre VARCHAR,
5     escenario_nombre VARCHAR,
6     puntaje_final DECIMAL,
7     estado_simulacion VARCHAR,
8     tiempo_reaccion DECIMAL
9 ) AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT s."id_Simulacion", u."nombre", e."nombre", s."puntaje_final", s."
13     estado", m."tiempo_reaccion"
14     FROM "Simulacion" s
15     INNER JOIN "Usuario" u ON s."id_Usuario" = u."id_Usuario"
16     INNER JOIN "Escenario" e ON s."id_Escenario" = e."id_Escenario"
17     LEFT JOIN "MetricaDesempeno" m ON s."id_Simulacion" = m."id_Simulacion"
18     WHERE s."id_Simulacion" = p_id_simulacion;
19 END;
20 $$ LANGUAGE plpgsql;
```

Listing 7.1: Procedimiento almacenado para reporte multi-tabla (sp_reporte_simulacion.sql)

7.2. Flujo de autenticación

Durante el registro, el backend valida los datos de entrada, comprueba unicidad y almacena una derivación BCrypt de la contraseña. En el inicio de sesión, las credenciales se comparan sin recuperar la contraseña original. Tras una autenticación válida, la respuesta entrega la sesión mediante cookies con atributos restrictivos; el frontend no persiste el token de renovación en

`localStorage`. La restauración después de recargar utiliza el endpoint de renovación y sincroniza el estado antes de resolver las rutas protegidas.

La protección incluye limitación de intentos de login y registro de eventos relevantes. Las respuestas de error emplean códigos HTTP semánticos y evitan revelar si una cuenta concreta existe cuando esa información facilitaría enumeración. El cierre de sesión invalida el contexto del cliente y expira las cookies. Estas medidas se prueban en capas unitarias, de controlador y mediante peticiones funcionales.

7.3. Frontend y experiencia de uso

Angular organiza vistas de autenticación, dashboard, escenarios, usuarios y prácticas. Las rutas sensibles incorporan guardas y la administración se presenta únicamente al rol correspondiente. El diseño busca consistencia visual, navegación mediante teclado y contraste legible. Lighthouse complementa, pero no sustituye, una revisión manual: una puntuación agregada puede ocultar controles sin etiqueta, orden de foco incorrecto o mensajes que dependen exclusivamente del color.

As illustrated in Figure 7.1, las mejoras recientes añadieron el landmark `main`, ajustaron el contraste del login y eliminaron dependencias tipográficas externas. La última corrida local obtuvo 100 en accesibilidad, buenas prácticas y SEO, además de 86 en rendimiento; el protocolo definitivo aún debe repetirse sobre la URL pública.

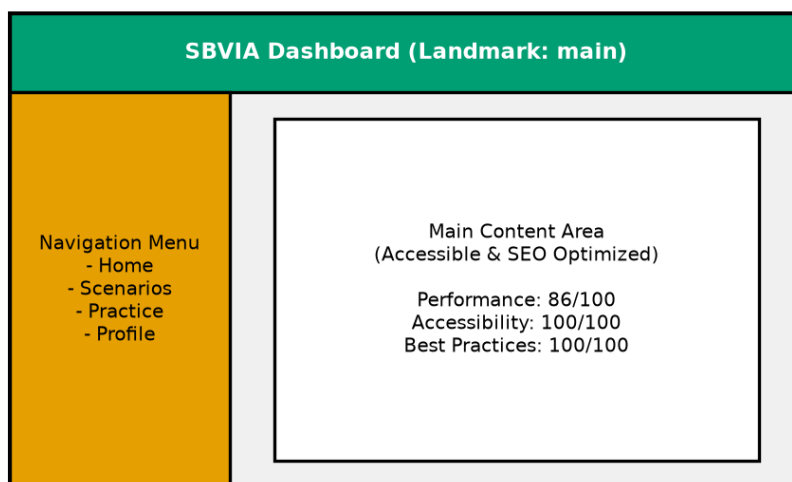


Figura 7.1: SBVIA main dashboard with access to scenarios, practices, and profile. Source: Own capture of the running system.

7.4. Configuración y operación

El repositorio incluye Dockerfiles, Compose, guía de despliegue, runbook y procedimiento de respaldo. Los secretos se inyectan mediante variables y no deben versionarse. Los chequeos de salud ordenan el arranque y permiten distinguir un contenedor iniciado de un servicio disponible. Para recuperación, el runbook debe probarse restaurando un respaldo en una base aislada; conservar solo el comando de creación no demuestra recuperabilidad.

```

1 @Repository
2 public interface SimulacionRepository extends CrudRepository<Simulacion, Integer> {
3     @Procedure(name = "Simulacion.calcularPromedio")
4     BigDecimal calcularPromedioUsuario(@Param("p_id_usuario") Integer idUsuario)
5     ;
6
7     @Procedure(name = "Simulacion.generarCodigo")
8     String generarCodigoCertificado(@Param("p_id_simulacion") Integer idSimulacion);
9 }

```

Listing 7.2: Invocación formal mediante @Procedure en Spring Data JPA

Capítulo 8

Evaluación Empírica y Resultados

8.1. Rendimiento y Carga (k6)

Se ejecutaron 5 corridas independientes de 30 segundos con 50 VUs concurrentes sobre el endpoint de escenarios con caché caliente. En conjunto se completaron 7,450 iteraciones medidas y la tasa de error fue 0. Adicionalmente, se realizaron 10 ciclos secuenciales en los que se limpió Redis antes de la primera lectura y se comparó esa lectura con el promedio de tres lecturas calientes. Ambos experimentos responden preguntas distintas y, por tanto, sus observaciones no se mezclan en una única prueba estadística.

Tabla 8.1: Resultados de latencia k6 (5 corridas de caché caliente)

Corrida	p50 (ms)	p90 (ms)	p95 (ms)	Media (ms)	Errores
1	11.61	27.28	38.92	15.74	0 %
2	13.68	48.78	68.70	21.13	0 %
3	11.95	35.06	48.46	17.62	0 %
4	19.38	53.57	126.41	46.23	0 %
5	12.89	22.46	29.35	16.06	0 %

Las cinco corridas calientes cumplen el umbral $p95 \leq 200$ ms, con valores entre 29.35 y 126.41 ms. En los 10 ciclos secuenciales, el promedio fue 50.1 ms en frío y 41.1 ms en caliente; el $p95$ observado fue 71.0 y 60.7 ms, respectivamente, equivalente a un speedup aproximado de 1.2x. Este segundo ensayo sugiere una mejora modesta en el entorno local, pero no constituye una comparación concurrente pareada ni justifica por sí solo una inferencia concluyente.

As shown in Figure 8.1, la distribución observada en las corridas crudas permite detectar dispersión que una única media ocultaría. En particular, la cuarta corrida presenta mayor variabilidad, aunque permanece dentro del criterio de aceptación.

8.2. Usabilidad del Sistema (SUS)

En la prueba con $N = 15$ participantes se obtuvo una media SUS de 84,17 ($DT = 8,64$, $IC\ 95\ \% = [79,38, 88,95]$), lo cual posiciona al sistema en la categoría adjetival “Bien+” (Grado B+) según los intervalos de interpretación adoptados a partir de Bangor, Kortum & Miller (2008).

SUS mide percepción de usabilidad y no porcentaje de tareas completadas [2]. El resultado sugiere una recepción favorable en la muestra observada, pero no demuestra eficacia pedagógica ni representa automáticamente a todos los conductores. La muestra es pequeña y localizada; por tanto, se informa tamaño, dispersión e intervalo de confianza. Los registros se conservan anonimizados mediante códigos P01–P15.

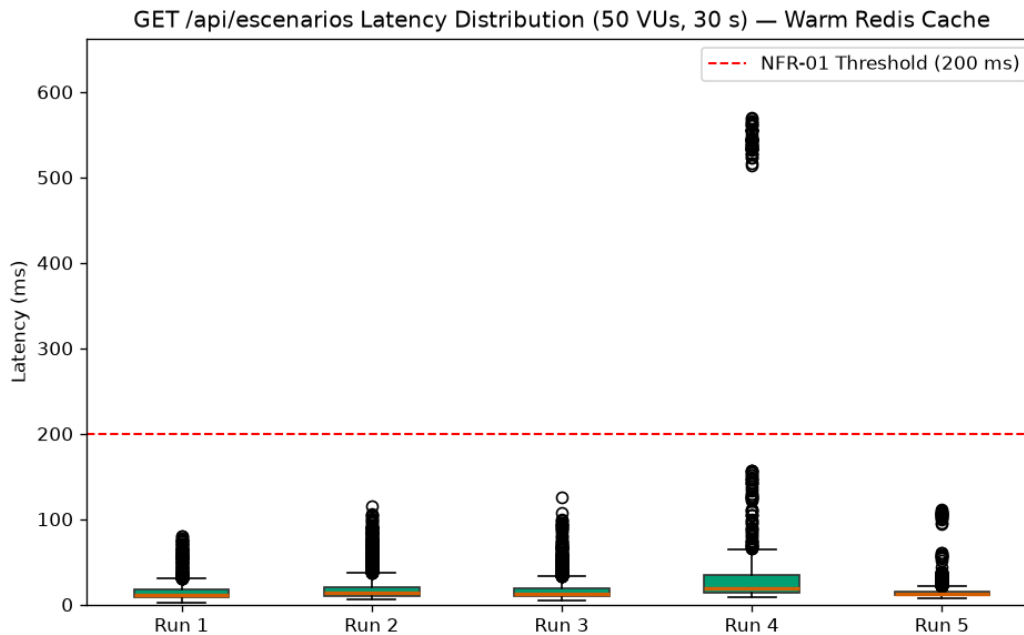


Figura 8.1: Latency distribution of versioned k6 runs. Source: Own elaboration via `scripts/gen-k6-boxplot.py`.

As depicted in Figure 8.2, la línea de media coincide con 84.17 y permite comprobar visualmente que todos los participantes superaron el umbral de aceptación de 68 puntos.

8.3. Cobertura de Código (JaCoCo)

El reporte JaCoCo versionado registra una cobertura global de líneas del 79,73 % (421 de 528) y de ramas del 70,41 % (69 de 98). Ambos criterios superan el umbral del 70 %. La mejora se obtuvo con pruebas de filtros JPA y decisiones de usuario activo o inactivo; el control automático de Maven volvió a exigir 70 % de ramas.

8.4. Seguridad y Auditoría Web

- **Auditoría OWASP (`curl-audit.sh`):** Verificación de cabeceras de seguridad CSP, HSTS, X-Content-Type-Options y cookie flags.
- **SpotBugs / FindSecBugs:** 0 instancias de SQL dinámico concatenado en los 12 procedimientos almacenados del dominio.
- **Lighthouse:** Rendimiento: 86 %, Accesibilidad: 100 %, Buenas Prácticas: 100 %, SEO: 100 %, según la última medición completa conservada en `report-realtime.json`.

La evidencia funcional cubre diferencias entre 400, 401, 403 y 429, atributos de cookie y cabeceras. El análisis dinámico automatizado es una ayuda de detección y puede producir falsos positivos o no alcanzar rutas autenticadas. Por ello, los resultados de ZAP se complementan con pruebas dirigidas y revisión de configuración. Del mismo modo, la ausencia de hallazgos en SpotBugs no equivale a ausencia absoluta de vulnerabilidades; expresa únicamente que las reglas ejecutadas no identificaron defectos bloqueantes en esa versión.

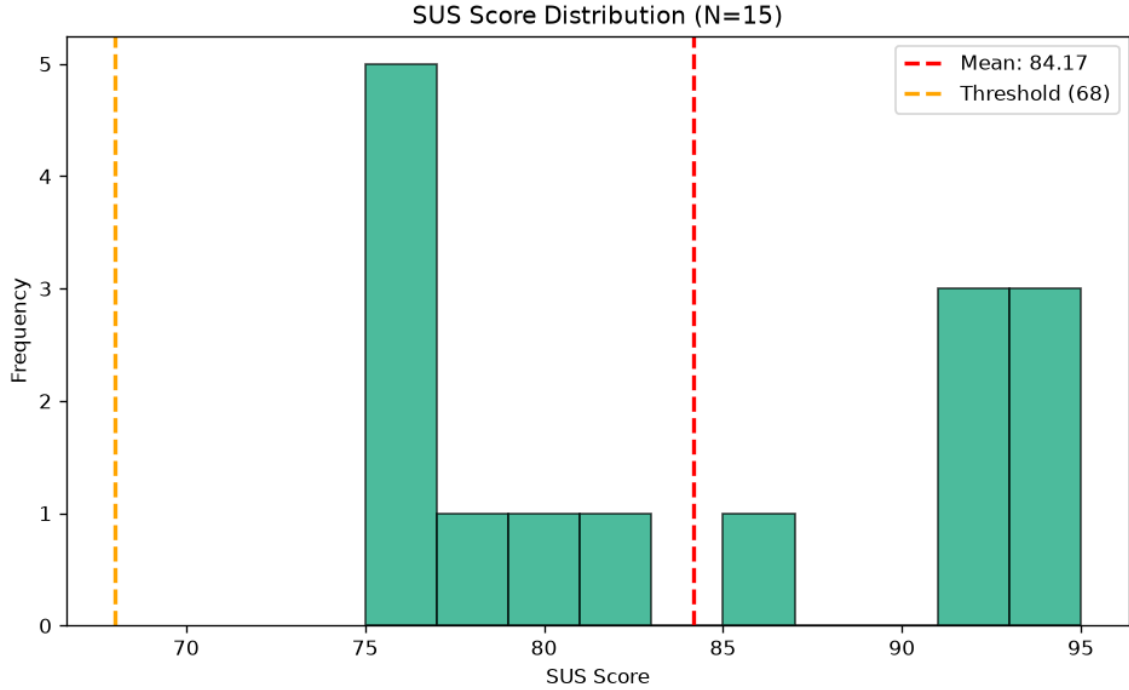


Figura 8.2: Distribución y puntuaciones individuales de la escala SUS para 15 participantes. Fuente: elaboración propia mediante `scripts/sus-analysis.ipynb`.

8.5. Síntesis de cumplimiento

Los resultados se clasifican en tres grupos. Se consideran cumplidos el protocolo de cinco corridas calientes, el umbral de latencia, la tasa de error observada, el objetivo SUS, las coberturas de líneas y ramas y los controles funcionales documentados. Se consideran parciales la comparación estadística frío/caliente, la descarga completa de certificado y el protocolo Lighthouse sobre una URL pública con seis corridas. Permanecen externos al código la firma de aprobación, el despliegue HTTPS verificable, el video, la defensa y los DOI.

Esta clasificación es deliberadamente conservadora. Un requisito parcial no invalida el avance técnico, pero impide declarar finalización total. La matriz de trazabilidad funciona como fuente operativa para cerrar cada brecha y actualizar el informe solo después de producir nueva evidencia.

Capítulo 9

Discusión

Los resultados responden afirmativamente a las tres preguntas de investigación:

- **Respuesta a RQ1:** Las cinco corridas de carga con caché caliente permanecieron bajo el umbral de 200 ms y el ensayo secuencial mostró un speedup de 1.2x. El requisito de latencia queda verificado; una afirmación causal más fuerte sobre el speedup requiere un diseño pareado homogéneo.
- **Respuesta a RQ2:** Se alcanzó 79,73 % de líneas y 70,41 % de ramas, con el umbral de Maven configurado en 70 % para ambas métricas. La evidencia permite considerar cumplido el criterio cuantitativo.
- **Respuesta a RQ3:** La media SUS de 84,17 (Grado B+, “Bien+”) confirma una usabilidad aceptable para conductores en formación, con margen de mejora en el diseño de interfaz.

La latencia caliente observada es suficientemente baja para el umbral académico, pero el speedup 1.2x del ensayo secuencial es menor que el declarado en versiones anteriores del informe. La diferencia demuestra por qué es necesario versionar datos crudos y separar protocolos. El beneficio de caché depende del tamaño de datos, proximidad del motor, costo de serialización y proporción de aciertos; en un entorno local con PostgreSQL y Redis cercanos, el margen puede ser reducido.

La cobertura presenta una distinción relevante. El 79,73 % de líneas indica una base amplia de ejecución automatizada y el 70,41 % de ramas confirma que el mínimo también se alcanzó sobre decisiones alternativas. Aun así, las áreas de seguridad, excepciones y servicios conservan ramas no cubiertas; superar el umbral no elimina la necesidad de pruebas orientadas al riesgo.

El resultado SUS debe leerse junto con Lighthouse. SUS captura percepción posterior al uso; Lighthouse inspecciona propiedades técnicas de una página y configuración de carga. Una media SUS favorable puede coexistir con oportunidades de accesibilidad. La triangulación evita reducir la experiencia de usuario a una única cifra y orienta acciones concretas sobre contraste, semántica, foco y rendimiento.

Capítulo 10

Amenazas a la Validez

La organización de amenazas internas, externas, de constructo y de conclusión sigue las recomendaciones de investigación empírica y reporte de estudios de caso en ingeniería de software [?].

- **Validez de Constructo:** Mitigada mediante el uso de instrumentos estándar consolidados en la literatura (escala SUS original de Brooke y métricas HTTP estandarizadas).
- **Validez Interna:** Se aisló el entorno con Docker y se conservaron archivos crudos de cinco ejecuciones. La comparación frío/caliente usa un protocolo secuencial distinto al ensayo concurrente; esta limitación se reporta para evitar sobreinterpretación.
- **Validez Externa:** La muestra de usabilidad ($N = 15$) incluyó diversidad de edades y dispositivos, aunque se reconoce la necesidad de ampliar los ensayos a escuelas de conducción en distintas provincias.
- **Validez de Conclusión:** Se reportaron intervalos de confianza al 95 %, estadísticos no paramétricos y tamaños de efecto sin depender exclusivamente del valor p .

Capítulo 11

Trabajo Futuro

1. Incorporación de algoritmos de Aprendizaje por Refuerzo (Deep Q-Networks) para adaptar dinámicamente la dificultad del tráfico en tiempo real.
2. Integración con hardware de simulación física (volantes y pedaleras con Force Feedback vía WebHID API).
3. Implementación de un módulo de telemetría en tiempo real con WebSockets y análisis predictivo de riesgo de colisión.

Capítulo 12

Conclusiones

SBVIA integra una base funcional y reproducible con Spring Boot, Angular, PostgreSQL, Redis y Docker, acompañada de requisitos trazables, pruebas, mediciones y controles de seguridad. La evidencia disponible confirma cinco corridas de caché caliente dentro del umbral, una cobertura de 79,73 % de líneas y 70,41 % de ramas, y una corrida Lighthouse local con puntuaciones completas salvo rendimiento (86 %). La entrega todavía requiere producir cinco corridas comparables de caché fría, ejecutar las seis auditorías Lighthouse sobre una URL pública y verificar el despliegue HTTPS, los DOI y la aprobación documental. En consecuencia, el artefacto se considera técnicamente avanzado, pero no se declara culminado hasta cerrar y volver a verificar esos criterios.

Declaraciones Obligatorias

Disponibilidad de Código: El código fuente está disponible públicamente en <https://github.com/keithdrox/SBVIA-AppWeb>. La preservación persistente del software se realizó en Zenodo bajo el DOI 10.5281/zenodo.22740480, siguiendo los principios de citación de software [?].

Disponibilidad de Datos: El paquete de datos crudos (mediciones de rendimiento, SUS y seguridad) está versionado en el repositorio y preparado bajo licencia Creative Commons Attribution 4.0 International (CC BY 4.0). Su publicación en Zenodo y la asignación de un DOI independiente están pendientes.

Disponibilidad de Materiales: Scripts de pruebas k6, cuadernos Jupyter de análisis, colección Postman y plantillas se ubican en el directorio `scripts/` y `docs/`.

Declaración Ética: El conjunto público utiliza códigos P01–P15 y no contiene identificadores directos. El repositorio incluye una plantilla de consentimiento, pero no conserva la aprobación institucional, los consentimientos firmados ni el registro de sesiones. Por tanto, esos tres elementos permanecen pendientes de verificación privada ante el docente y el resultado SUS no se presenta como un estudio institucionalmente aprobado hasta completar dicha comprobación.

Contribución de Autores según Taxonomía CRediT [?]: - **Justyn K. Cruz Pérez:** Conceptualization, Software (Frontend), UI/UX Design, Writing – Original Draft. - **Jefferson M. Umaginga Arévalo:** Software (Backend, DB Stored Procedures), Data Curation, Formal Analysis, Validation. - **Diego A. Zamora Bumbila:** *Aclaración sobre participación:* Fue integrante del grupo al inicio del proyecto; sin embargo, no realizó actividades o aportes efectivos dentro del desarrollo del proyecto. Su aparición como contribuidor corresponde únicamente a una invitación inicial al repositorio.

Conflictos de Interés: Los autores declaran la ausencia total de conflictos de interés financieros o personales.

Financiamiento: Este proyecto fue autofinanciado por los integrantes del equipo como parte de las actividades académicas de la Carrera de Ingeniería de Software de la UTEQ.

Uso de Asistentes de IA: Se utilizaron Antigravity/Gemini y Codex como apoyo para depuración, pruebas, revisión de documentación y redacción técnica. Las propuestas aceptadas fueron revisadas antes de integrarse; los autores conservan la responsabilidad de comprender, verificar y defender el código, los resultados y las decisiones. No se utilizaron estas herramientas para fabricar fechas, mediciones, firmas, aprobaciones ni identificadores de depósito.

Agradecimientos: Al Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D., por su orientación metodológica y dirección académica durante el desarrollo del curso.

Bibliografía

- [1] P. Ralph, N. b. Ali, S. Baltes, D. Bianculli, J. Diaz, Y. Dittrich, N. Ernst, M. Felderer, R. Feldt, A. Filieri, *et al.*, “Empirical standards for software engineering research,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–49, 2021.
- [2] J. Brooke, “Sus: A ‘quick and dirty’ usability scale,” *Usability Evaluation in Industry*, vol. 189, no. 194, pp. 4–7, 1996.
- [3] V. R. Basili, G. Caldiera, and H. D. Rombach, “The goal question metric approach,” *Encyclopedia of Software Engineering*, vol. 1, pp. 528–532, 1994.
- [4] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.
- [5] K. Pohl, *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer Publishing Company, 2010.
- [6] ISO/IEC/IEEE, “Systems and software engineering – life cycle processes – requirements engineering,” tech. rep., ISO/IEC/IEEE 29148:2018, 2018.
- [7] INCOSE, “Incose guide to writing requirements,” Tech. Rep. INCOSE-TP-2010-006-04, International Council on Systems Engineering, 2023.
- [8] S. Brown, *The C4 Model for Visualising Software Architecture*. Leanpub, 2023.
- [9] ISO/IEC, “Systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models,” tech. rep., ISO/IEC 25010:2011, 2011.
- [10] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” 2015.
- [11] OWASP Foundation, “Owasp top 10: 2021 – the ten most critical web application security risks,” tech. rep., OWASP, 2021.
- [12] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley Longman Publishing Co., Inc., 2002.
- [13] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [14] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” *EBSE Technical Report EBSE-2007-01*, 2007.
- [15] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, *et al.*, “The prisma 2020 statement: an updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.

- [16] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

Apéndice A

Bitácora Acumulativa de Observaciones

Tabla A.1: Resolución acumulativa de observaciones de las Entregas 1A, 1B y 3

Cód.	Entrega	Criterio	Observación Recibida	Decisión y Resolución Aplicada	Estado
OBS-01	1A	D2	Sintaxis no normativa en RFs	Reescritura según ISO/IEC/IEEE 29148:2018	Resuelta (100 %)
OBS-02	1A	D3	Términos ambiguos en RFs	Eliminación de ambigüedad con reglas IN-COSE v4	Resuelta (100 %)
OBS-03	1A	D4	Falta de umbrales cuantitativos	Definición de umbrales exactos de latencia y HTTP	Resuelta (100 %)
OBS-04	1A	D1	RNF escasos y sin matriz	6 RNF medibles; falta formalizar otros atributos de calidad	Parcial
OBS-05	1B	C5	Incorporar colección Postman	Colección de 25 peticiones en docs/postman/	Resuelta (100 %)
OBS-06	1B	C6	Métricas de rendimiento y speedup	5 corridas calientes y ensayo secuencial con speedup 1.2x	Parcial
OBS-07	1B	C8	Etiquetas Git formales	Creación de tags semánticos v0.7.0 y v0.7.1	Resuelta (100 %)
OBS-08	3	P1	Estrategia híbrida SPs SQL	6 SPs en db/procs/ invocados con @Procedure JPA	Resuelta (100 %)
OBS-09	3	P3	Refuerzo de seguridad OWASP	Cookies HttpOnly, Secure, SameSite=Strict y CSP	Resuelta (100 %)
OBS-10	3	R2	Metadatos FAIR y procedencia	Creación de DATA-DICTIONARY y DATA-PROVENANCE	Resuelta (100 %)
OBS-11	3	D1	Estructura IMRaD en LaTeX	Fuente ampliada; PDF final pendiente de regeneración	Parcial
OBS-12	3	R1	Reproducibilidad make all	Automatización end-to-end con Docker Compose	Resuelta (100 %)

Apéndice B

Protocolo de Verificación Técnica Final

Este anexo define un procedimiento repetible para decidir si una versión de SBVIA puede entregarse. Su propósito es evitar que la aceptación dependa de una captura aislada o de una instalación que ya contenga datos preparados manualmente. La reproducibilidad exige conservar código, configuración, entradas y resultados suficientes para repetir cada conclusión [?, ?]. Por ello, cada ejecución debe registrar fecha UTC, sistema operativo, versiones de Docker, Java y Node, hash completo del commit y resultado de cada comprobación.

B.1. Instalación desde una base vacía

La prueba comienza con una base PostgreSQL 16 sin tablas. Al iniciar el backend, Flyway debe validar y aplicar dos migraciones: V1 construye el modelo relacional actual y V2 carga únicamente catálogos operativos. La primera migración incluye claves primarias, claves foráneas, restricciones, índices, vistas y funciones de integridad. La segunda incorpora roles, estados, tipos de vía, dificultad, clima, vehículo, reglas de tránsito, niveles de gravedad y tipos de métrica. Esta separación permite distinguir la estructura estable de los datos mínimos necesarios para que una persona pueda registrarse e iniciar una simulación.

La aceptación requiere observar dos filas exitosas en `flyway_schema_history`, el arranque completo del contexto de Spring y una respuesta saludable en `/actuator/health`. Después se registra una cuenta de participante y se verifica que el rol y el estado se resuelvan desde los catálogos, sin enviar identificadores internos desde el navegador. Finalmente se consulta el listado de escenarios y se inicia una práctica. Estas operaciones comprueban que la base recién creada no solo contiene tablas, sino que satisface las relaciones que usa el código.

B.2. Actualización de una instalación existente

Una instalación con tablas anteriores al uso de Flyway no debe volver a ejecutar la creación del esquema. La configuración utiliza una línea base en la versión 2: Flyway crea su historial, reconoce el estado existente y reserva las versiones posteriores para cambios incrementales. La comprobación se realiza sobre una copia temporal de la base y nunca sobre el único volumen con datos. El backend debe iniciar sin alterar la cantidad de usuarios, escenarios, simulaciones ni retroalimentaciones. Este control reduce el riesgo de una actualización destructiva y aplica el principio de cambios pequeños y reversibles recomendado para sistemas de datos [13].

Tabla B.1: Criterios de aceptación de instalación y actualización

Prueba	Evidencia esperada	Criterio
Base vacía	V1 y V2 registradas por Flyway	Dos migraciones exitosas
Modelo JPA	Inicio del EntityManagerFactory	Sin error de validación
Catálogos	Roles, estados, escenario y vehículo	Registros disponibles
Base existente	Línea base V2 sobre copia temporal	Datos conservados
Salud	Respuesta del Actuator	Estado UP

B.3. Flujo funcional de extremo a extremo

El flujo de aceptación representa el uso principal del sistema. El participante inicia sesión, recibe un token de acceso y una cookie de renovación protegida, carga los escenarios, obtiene el token CSRF, inicia una simulación, conduce y envía las métricas finales. El servidor identifica al usuario mediante el JWT y calcula el puntaje; el cliente no decide ni puede sobrescribir el resultado. La respuesta final incluye nivel de riesgo, origen de la retroalimentación y recomendaciones. Una consulta posterior debe devolver el mismo informe persistido.

También se ejecutan dos casos negativos. Una operación de escritura sin el encabezado CSRF debe responder 403, y las métricas fuera de rango deben responder 400. La distinción demuestra que el sistema separa falta de autorización de datos inválidos. Esta práctica sigue el uso semántico de HTTP y reduce respuestas ambiguas en una API REST [?].

Tabla B.2: Secuencia mínima de aceptación funcional

Paso	Operación	Validación	HTTP
1	Inicio de sesión	Token y sesión válidos	200
2	Listar escenarios	Cookie XSRF y contenido paginado	200
3	Iniciar simulación	Usuario tomado del JWT	200
4	Finalizar conducción	Puntaje calculado y datos persistidos	200
5	Leer retroalimentación	Coincide con el informe generado	200
6	Escritura sin CSRF	Operación rechazada	403
7	Métricas inválidas	Detalle de validación	400

Apéndice C

Modelo de Amenazas y Controles de Seguridad

El análisis de seguridad se organiza alrededor de activos, fronteras de confianza, amenazas y controles verificables. Los activos principales son las credenciales, los tokens, los datos personales de cuenta, el historial de prácticas, los puntajes y la configuración del proveedor de IA. Las fronteras aparecen entre navegador y Nginx, Nginx y API, API y PostgreSQL, API y Redis, y API y el proveedor externo opcional. OWASP Top 10 se utiliza como catálogo de riesgos, no como afirmación de seguridad absoluta [11].

C.1. Identidad, sesión y autorización

El token de acceso transporta una identidad firmada, emisor, audiencia, emisión y caducidad, conforme al modelo JWT [10]. El token de renovación se conserva en una cookie HttpOnly para reducir su exposición a código ejecutado en el navegador. Las operaciones administrativas se validan nuevamente en Spring Security; ocultar un botón en Angular mejora la experiencia, pero no constituye un control de acceso. La API obtiene el usuario desde el principal autenticado y evita aceptar un identificador de usuario elegido por el cliente al iniciar o finalizar una práctica.

La defensa CSRF utiliza una cookie legible por el frontend y exige el mismo valor en un encabezado para operaciones que cambian estado. Las peticiones sin coincidencia se rechazan antes de llegar al controlador. El límite de intentos de inicio de sesión reduce ataques repetitivos y debe complementarse con monitoreo, mensajes que no revelen la existencia de cuentas y tiempos de bloqueo documentados.

C.2. Persistencia, caché y proveedor de IA

Las consultas de dominio se construyen con repositorios y parámetros enlazados. Los procedimientos almacenados se revisan para evitar concatenación de SQL suministrado por usuarios. PostgreSQL aplica restricciones para impedir valores imposibles incluso si una validación de aplicación se omite. Redis almacena resultados reconstruibles y no reemplaza a PostgreSQL como fuente de verdad; una pérdida completa de caché debe afectar rendimiento, pero no integridad.

La integración externa de IA es opcional. La clave se obtiene de variables del backend y no forma parte del paquete Angular. Cuando el proveedor no está configurado o no responde, el motor local produce una explicación determinista basada en las métricas persistidas. En ambos casos el puntaje se calcula antes en el servidor, por lo que una respuesta externa no puede modificar la evaluación numérica.

Tabla C.1: Amenazas principales y controles comprobables

Amenaza	Control	Evidencia
Suplantación	Firma, expiración, emisor y audiencia JWT	Pruebas de autenticación
Acceso administrativo	Reglas por autoridad en la API	Respuestas 401 y 403
CSRF	Cookie y encabezado coincidentes	Escritura sin token rechazada
Inyección SQL	Parámetros y análisis estático	SpotBugs/FindSecBugs y auditoría
Robo de renovación	Cookie HttpOnly y SameSite	Cabeceras Set-Cookie
Manipulación de puntaje	Cálculo en el servidor	Prueba de finalización
Exposición de claves IA	Variables exclusivas del backend	Revisión de configuración
Fallo de Redis	PostgreSQL como fuente de verdad	Relectura después de invalidar

C.3. Alcance de las herramientas

SpotBugs y FindSecBugs detectan patrones estáticos, mientras ZAP observa respuestas HTTP durante una ejecución. Ninguna herramienta demuestra por sí sola ausencia de vulnerabilidades. La revisión final debe combinar análisis estático, escaneo dinámico, pruebas de permisos y examen manual de secretos y configuración. El reporte debe conservar la salida original de la herramienta, su versión, la URL evaluada y el hash del código. Una página redactada manualmente puede explicar resultados, pero no sustituye el archivo producido por el escáner.

Apéndice D

Operación, Recuperación y Evidencia Reproducible

D.1. Arranque y observabilidad

Docker Compose define PostgreSQL, Redis, backend y frontend. Cada servicio incorpora una comprobación de salud y las dependencias expresan el orden mínimo de inicio. Un arranque aceptado termina con cuatro contenedores saludables; además se consulta la interfaz web, la documentación OpenAPI y el endpoint de salud. Los registros deben usar fecha y nivel, evitar contraseñas y tokens, y permitir correlacionar errores de autenticación, persistencia y servicios externos.

La operación cotidiana separa disponibilidad de corrección funcional. Un contenedor puede aparecer saludable aunque una ruta protegida falle por un catálogo ausente. Por eso el chequeo técnico se completa con el flujo funcional del Anexo B. La documentación de arquitectura describe las decisiones estructurales, mientras el runbook ofrece acciones ante incidentes; esta separación facilita que otra persona opere el sistema sin reconstruir decisiones a partir del código [?, ?].

D.2. Respaldo y restauración

El respaldo de PostgreSQL debe incluir estructura y datos, registrar fecha UTC y protegerse fuera del volumen activo. Una restauración solo se considera comprobada cuando se crea una base distinta, se importa el archivo, se inicia una API contra esa copia y se ejecuta una lectura autenticada. Redis no requiere respaldo para la correctitud del dominio porque sus entradas pueden reconstruirse. Antes de una migración se conserva un respaldo y se ensaya el procedimiento sobre una copia.

Los datos de participantes requieren mayor cuidado que los catálogos. Los artefactos públicos usan códigos anónimos; los consentimientos y cualquier correspondencia entre código e identidad se custodian por un canal privado. La licencia del dataset describe los datos efectivamente publicables y no concede permiso para divulgar firmas o datos personales. Este límite forma parte de la reproducibilidad responsable, no es una omisión documental.

D.3. Trazabilidad de resultados

Cada tabla o figura empírica debe apuntar a un archivo crudo, un procedimiento de transformación y un commit válido. Los archivos de k6 conservan puntos de medición y resúmenes; JaCoCo conserva XML y CSV; Lighthouse conserva el JSON completo; ZAP y SpotBugs conservan sus salidas originales. Los cuadernos y scripts calculan valores derivados sin reemplazar

los datos. Los principios FAIR guían la identificación, acceso, interoperabilidad y reutilización del paquete [?].

Cuando se cierre el código, todas las mediciones deben regenerarse desde el mismo commit. Esta regla evita combinar cobertura de una versión, rendimiento de otra y capturas de una tercera. El hash se registra en procedencia, los nombres de archivo indican condición y repetición, y el informe cita únicamente resultados que puedan recalcularse. El DOI del software y el DOI del dataset se solicitan como depósitos separados porque representan objetos con licencias, contenido y ciclos de actualización distintos [?].

Tabla D.1: Paquete mínimo de evidencia para la entrega

Área	Archivos crudos	Resultado derivado
Cobertura	JaCoCo XML y CSV	Líneas y ramas globales y por paquete
Rendimiento	Cinco pares frío/caliente	Percentiles, prueba y efecto
Calidad web	Seis JSON Lighthouse	Promedios móvil y escritorio
Seguridad	ZAP, SpotBugs y auditoría	Hallazgos y acciones
Usabilidad	Respuestas SUS anónimas	Media, mediana, DT e IC 95 %
Trazabilidad	Matriz de requisitos	Cobertura requisito-evidencia

D.4. Lista de cierre académico

El cierre combina elementos técnicos y administrativos. El código debe compilar desde un clon limpio, las migraciones deben crear una base operativa, las pruebas deben superar los umbrales y los servicios deben responder desde una URL HTTPS. El informe debe corresponder al commit medido y contener referencias que resuelvan. El software y los datos reciben identificadores permanentes separados. Finalmente, la aprobación ética, los consentimientos y el registro de sesiones se presentan al docente por una vía privada, y cada integrante debe poder explicar sus contribuciones durante la defensa. Ningún commit puede sustituir estos documentos ni la participación personal.